

Early Detection of Pancreatic Ductal Adenocarcinoma (PDAC) With Deep Learning

Sayandeep Dey¹, Aref Motamedi², Dr. Katie Ovens², Dr. Farhad Maleki²
1. Webber Academy 2. Dept. of Computer Science, University of Calgary
2023 Science Fair

Methodology and Procedure

Overall Work Flow Path

- Two datasets- healthy pancreases and another containing images of pancreases with cancer
- Two models: Segmentation Model and Classification Model. They both work in unison as a complete system
- Segmentation model trained by images from healthy dataset were used to train the segmentation model
- Segmentation all images.
- The segmentation model was a prebuilt U-net type model called resnet34
- Segmented model output is input to Classification model
- The data was split into two distinct sets: 80% of the original data went to the 'training set' and the remaining 20% went to the 'validation set'
- Classification model is based on Pre-built CNN model called googlenet
- The final output of the classification model was its accuracy

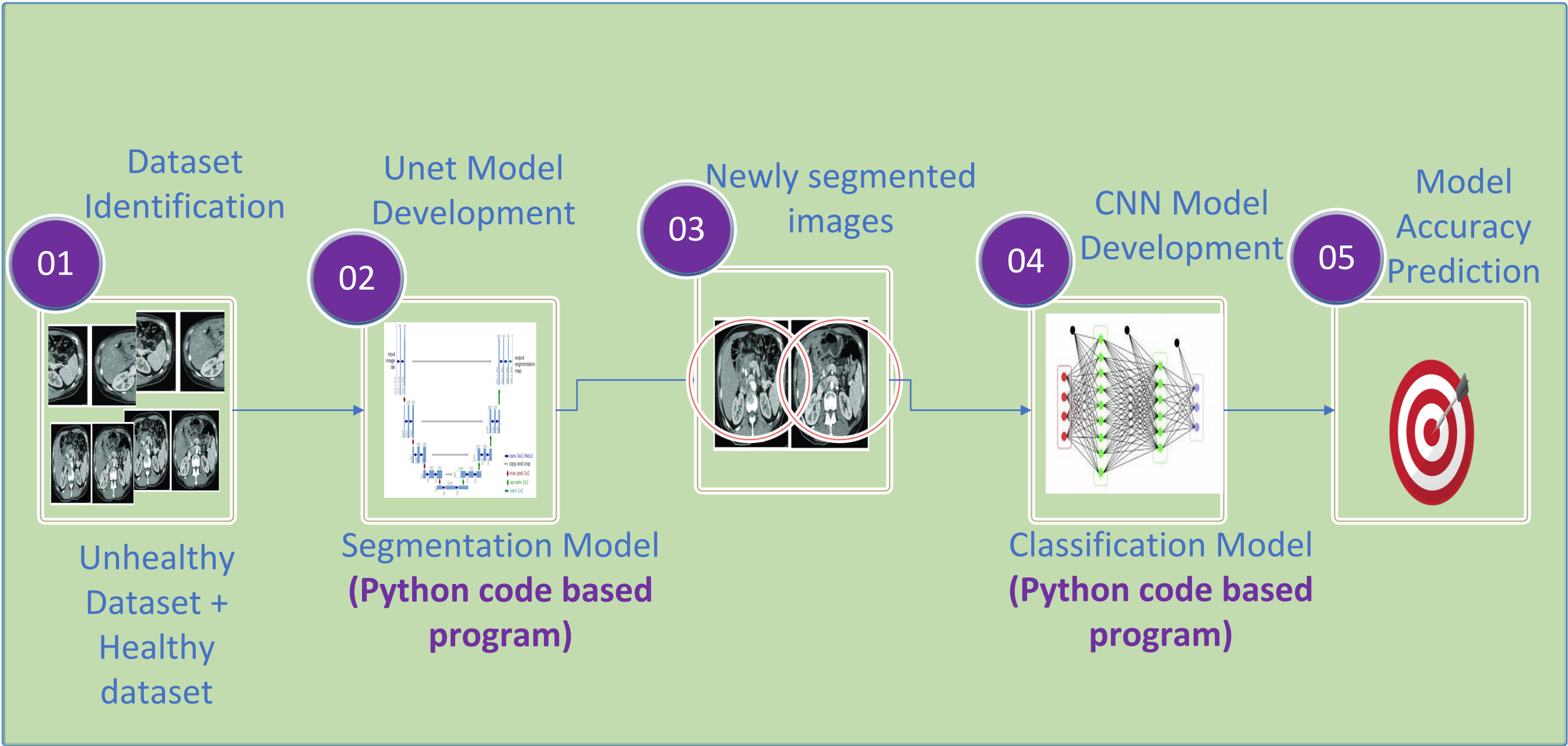


Figure 5. The Overall methodology and procedure followed to develop Segmentation model and Classification model and their training and validation process.

Image Segmentation Model

- Raw Data Collection**
- Data Processing**
- The dataset containing all healthy images of the pancreas was used as an input to the segmentation model
 - Data imported to a csv file and into the program and then extracted into the various subfolders. The data within all the subfolders was then processed(read) using SimpleITK
 - Three main functions were used: "Read_image" to read the image from the provided path, "Visualize" to plot or show the image, and "Dice_metric" to measure the accuracy
 - Two transform functions were used: a general transformation function that flips and rotates images to improve diversity and a second function that creates a mask of an input image using the built-in transform function in PyTorch and the clip function in numpy.
- Raw Data Pre-Processing**

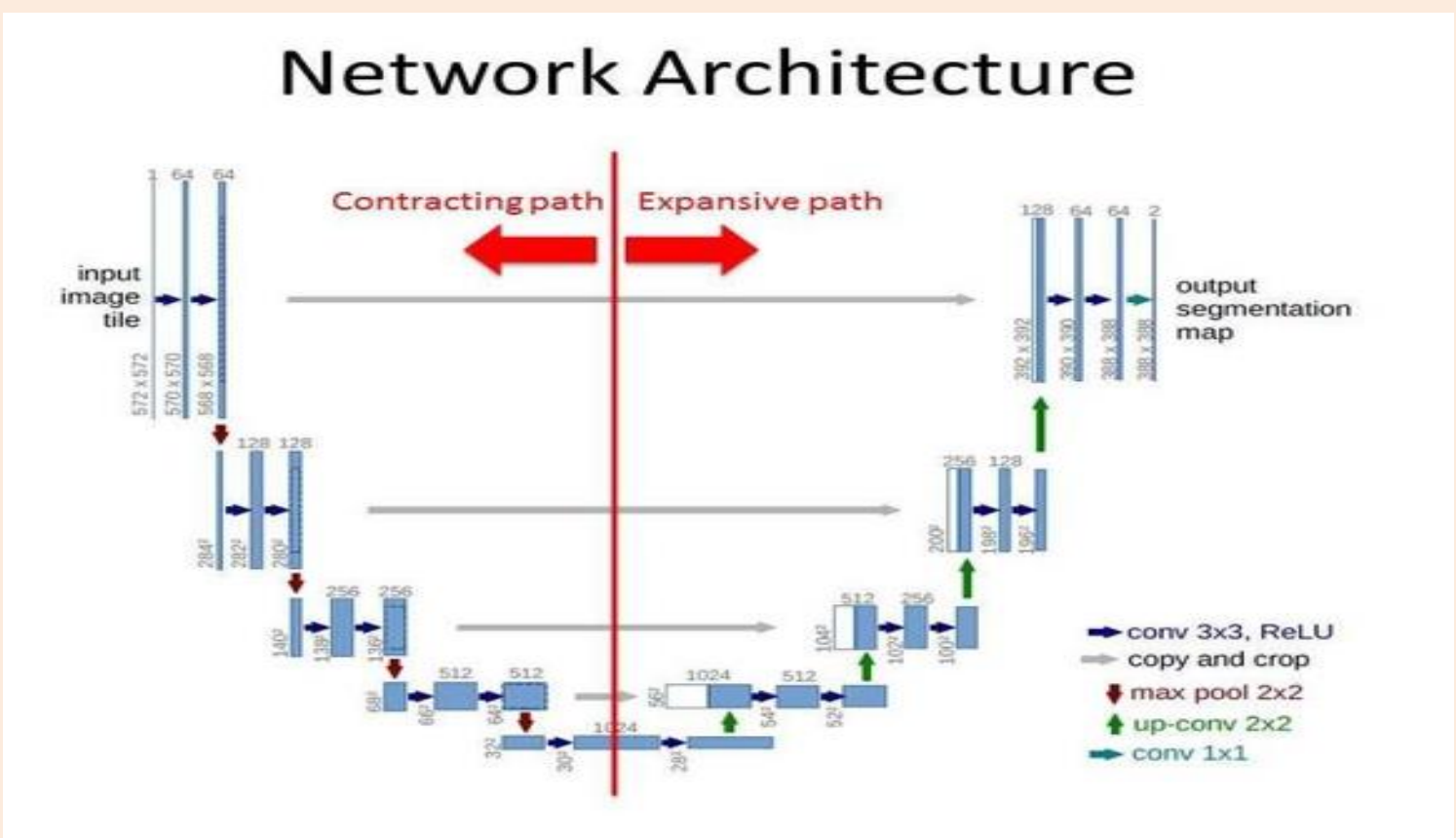


Figure 7. Segmentation Model Architecture

- Training Process**
- The model was trained for 15 epochs. The validation and training loss were measured at each epoch.
 - Three metrics were used to evaluate the model performance: validation loss, training loss, and dice coefficient.
 - Validation loss was calculated by comparing the predicted output to the actual output of the validation dataset and averaging the difference across all examples.
 - Training loss was calculated by comparing the predicted output to the actual output of the training dataset and averaging the difference across all examples.
 - The dice metric was measured using the "dice_metric" function.
 - Once the training was finished, all the images from both the healthy and unheleathy datasets were passed through the model to be segmented
- Model Training**
- Model Validation**

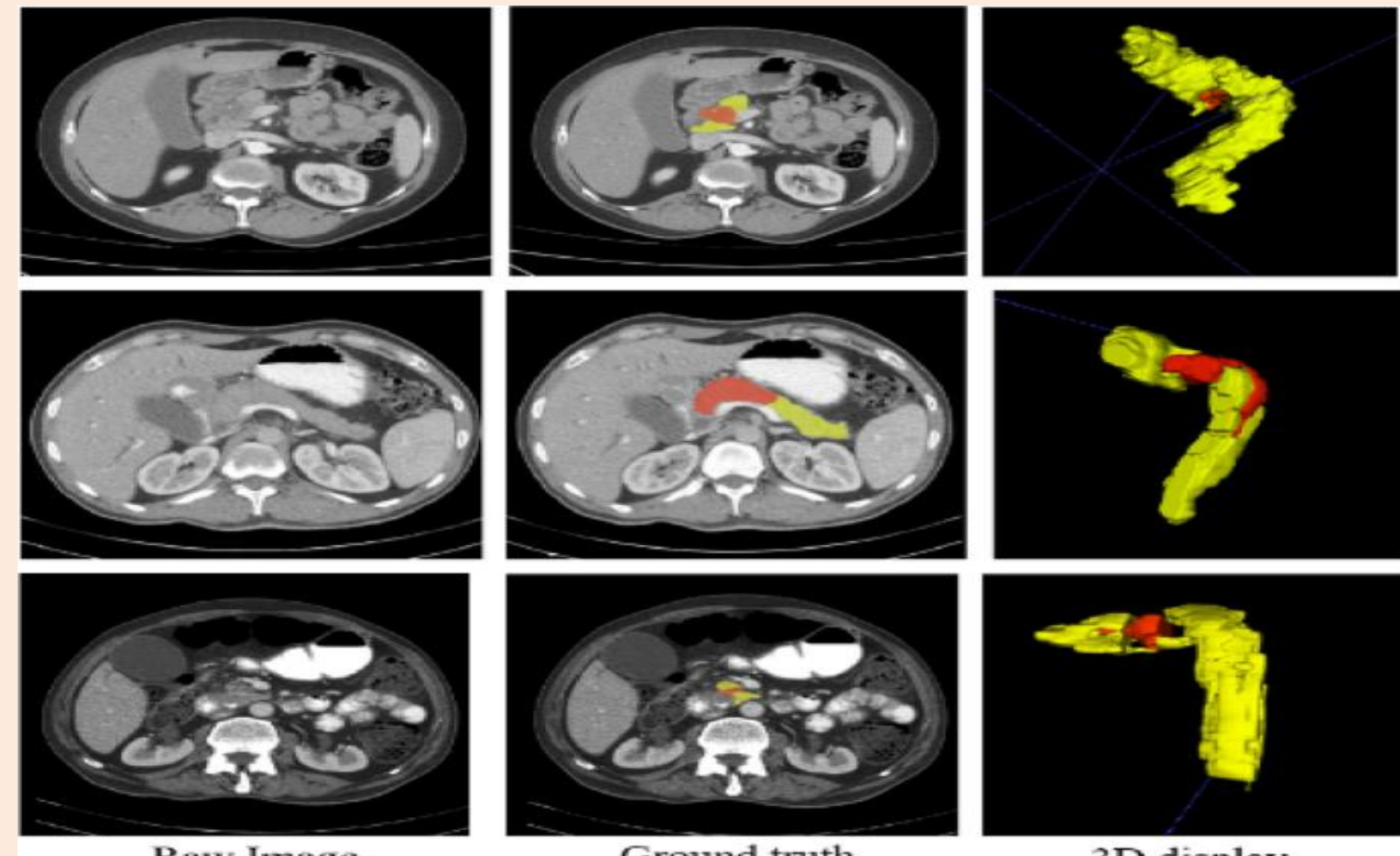


Figure 6. Process of Segmentation from raw CT images

Model Development

- A pre-trained U-net model named "resnet34" was used for the segmentation model.
- ResNet34 is a variant of the ResNet architecture with 34 layers which uses residual blocks consisting of two convolutional layers with batch normalization and ReLU activation, followed by an identity mapping
- To optimize accuracy, an encoder weight ("imagenet") is used, and the model is trained upon the GPU of the computer to optimize training time.
- Pre-trained ImageNet weights are used as a starting point for training the network on a specific task, allowing ResNet to leverage prior knowledge efficiently.
- The optimizer used for the model was 'adam'

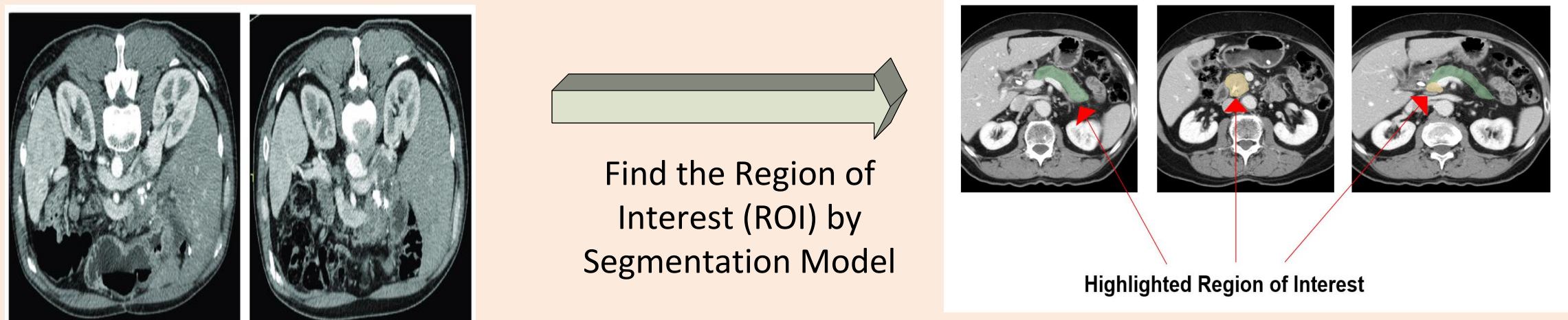


Figure 8. Segmentation Model Functional Representation

Classification Model

- Data Pre-Processing**
- The segmented images were collected and put into a csv file
 - The segmented images were then imported from the csv file and split into two distinct sets: 80% of the segmented images went into the training set and 20% went into the validation set
 - A classification model was created with the same functions and classes as the segmentation model, except for the transform functions
 - The images in both sets were shuffled to increase randomness and diversity.
 - The training set was used to train the classification model, while the validation set was used to validate the model after training
- Data Categorization**

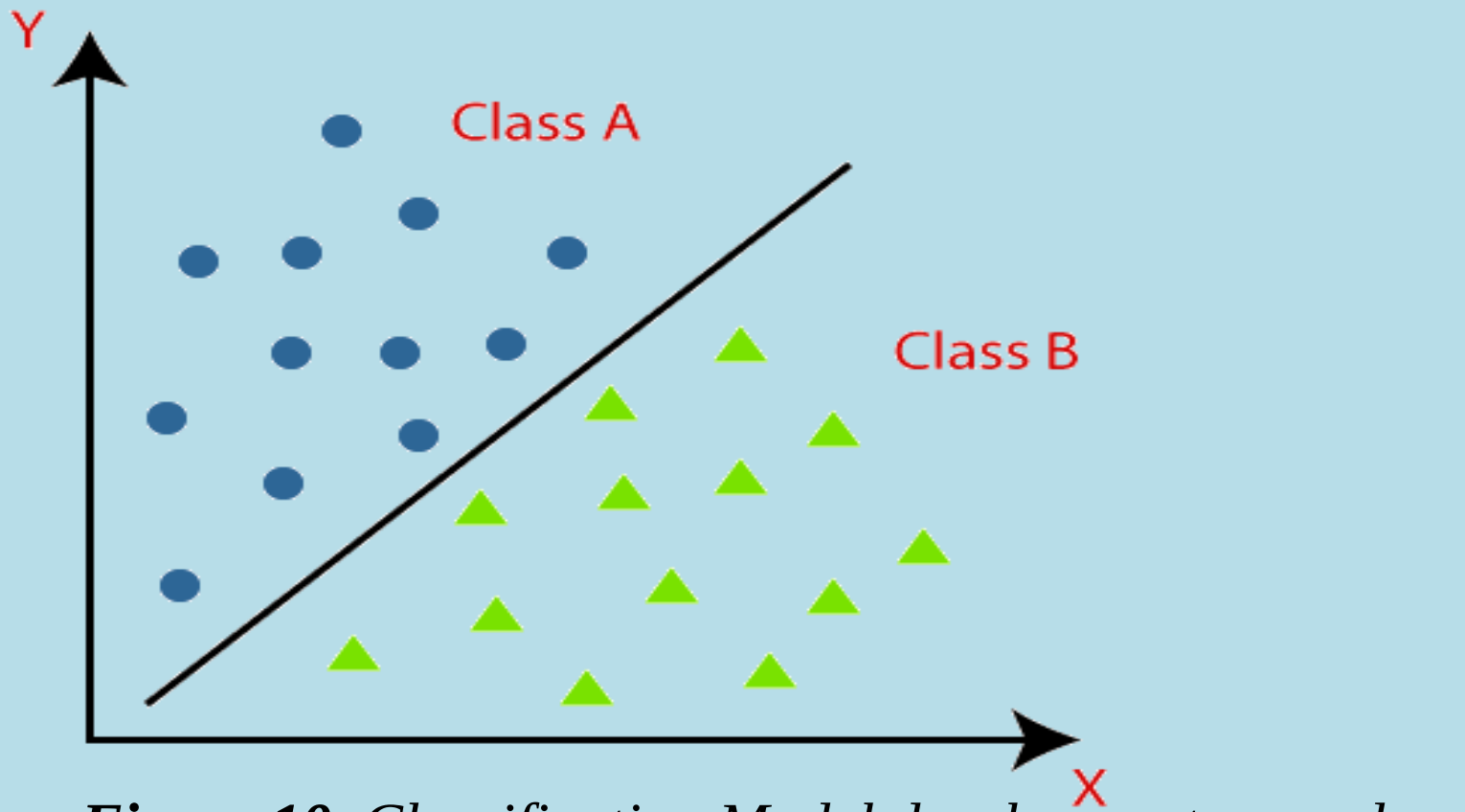


Figure 10. Classification Model development procedure

- Training Process**
- The model was trained upon the GPU for 100 epochs with an overfitting safety eject mechanism that shut down training if the model started to overfit within 5 epochs
 - If the model started to overfit within 5 epochs, the training would automatically shut down and the epoch with the best model accuracy and loss would be used
 - Validation loss was calculated by feeding the validation dataset through the trained model and comparing the predicted output to the actual output
 - Training loss was calculated by feeding the training dataset through the model and comparing the predicted output to the actual output.
- Model Training**
- Model Validation**

Model Development

- A pre-trained model, GoogleNet, was used for the classification model with weight "IMAGENET1K_V1" to optimize accuracy.
- GoogleNet CNN model consists of 22 layers including 9 Inception modules, which are a combination of various convolutions, pooling, and concatenations, allowing the network to capture features at multiple scales and resolutions in parallel
- GoogleNet used 1x1 convolutions to reduce the dimensionality of feature maps
- The pre-trained weights for IMAGENET1K_V1 were used as a starting point for transfer learning
- The weight was used to fine-tune pre-trained models on a new dataset with a different set of classes
- The loss was measured by CrossEntropyLoss and the optimizer used was 'adam'

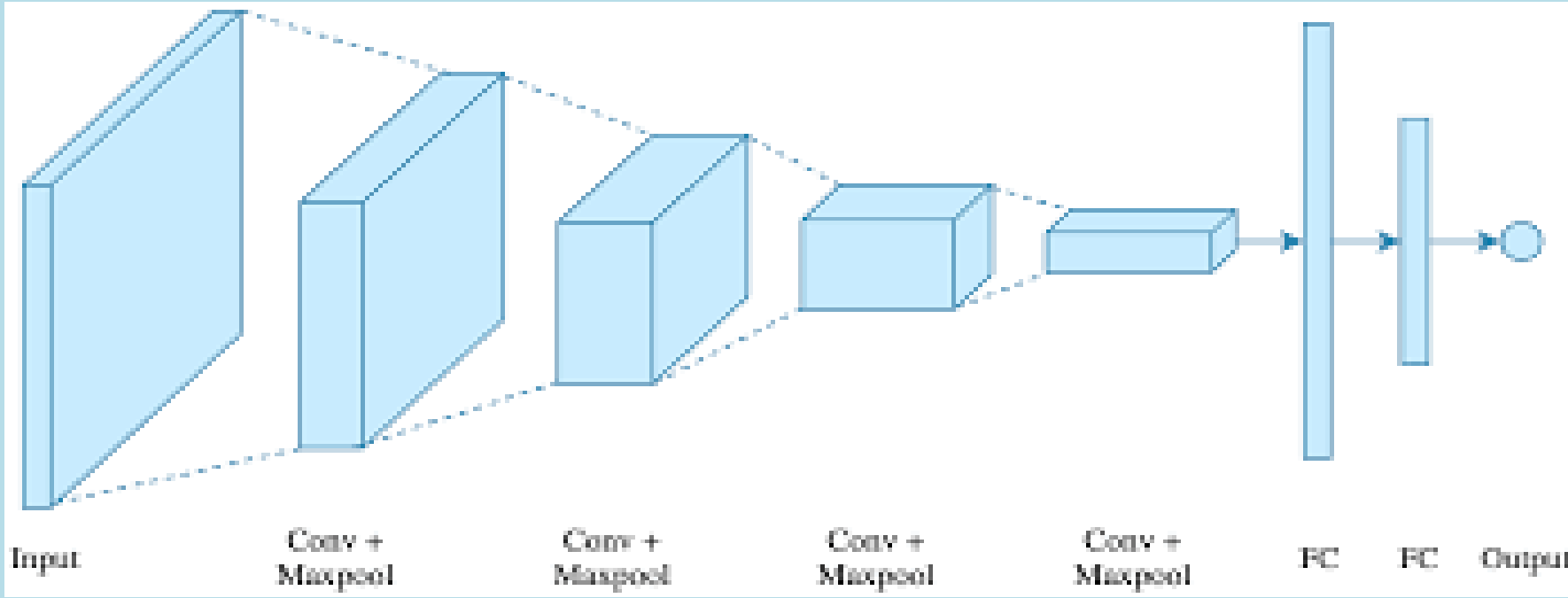


Figure 11. Classification Model overall flow

Samples of Segmentation Model Program Code

```
for epoch in range(num_epochs):
    losstotal = 0
    total = 0
    correct_pred = 0
    model.train()

    batch_idx = 0

    for inputs, targets in train_data_loader:
        total += 1
        inputs = inputs.to(device)
        targets = targets.to(device)

        optimizer.zero_grad()
        outputs = model(inputs.float())
        loss = criterion(outputs, targets.float())
        losstotal += loss * 1
        model.train()
        optimizer.step()

    if batch_idx < 10:
        with open('logbatches2vs.txt', 'a') as the_file:
            the_file.write(f'Batch: {batch_idx}, % Loss: {loss}\n')
        batch_idx += 1
```

```
general_transform = A.Compose([
    A.Resize(height=256, width=256),
    A.HorizontalFlip(p=0.5),
    A.Rotate(limit=90, p=0.5),
    A.Normalize(mean=[0.485, 0.455, 0.4], std=[0.229, 0.224, 0.225]),
])

train_data_loader = DataLoader(train_data_loader.dataset, batch_size=16, shuffle=True, num_workers=4)
val_data_loader = DataLoader(val_data_loader.dataset, batch_size=16, shuffle=False, num_workers=4)
```

Samples of Classification Model Program Code

```
class ImageDataset(Dataset):
    def __init__(self, normal_scans_path, abnormal_scans_path, image_transform, general_transform):
        self.normal_scans_path = normal_scans_path
        self.abnormal_scans_path = abnormal_scans_path
        self.image_transform = image_transform
        self.general_transform = general_transform

        self.normal_images_paths = []
        self.abnormal_images_paths = []

        self.normal_images_paths = sorted(glob.glob(normal_scans_path + '**/*.nii.gz'), recursive=True)
        self.abnormal_images_paths = sorted(glob.glob(abnormal_scans_path + '**/*.nii.gz'), recursive=True)

        self.labels = [0 for i in range(len(normal_images_paths))] + [1 for i in range(len(abnormal_images_paths))]

        self.images_paths = self.normal_images_paths + self.abnormal_images_paths
        self.labels = self.labels + self.labels

        self.indexables = list(zip(self.images_paths, self.labels))

        random.shuffle(self.indexables)
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

model = torchvision.models.googlenet(weights=torchvision.models.googlenet_weights.IMAGENET1K_V1)
num_ftrs = model.fc.in_features
model.fc = torch.nn.Linear(num_ftrs, 2)

num_gpus = torch.cuda.device_count()

if num_gpus > 1:
    model = torch.nn.DataParallel(model)

model = model.to(device)

criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```